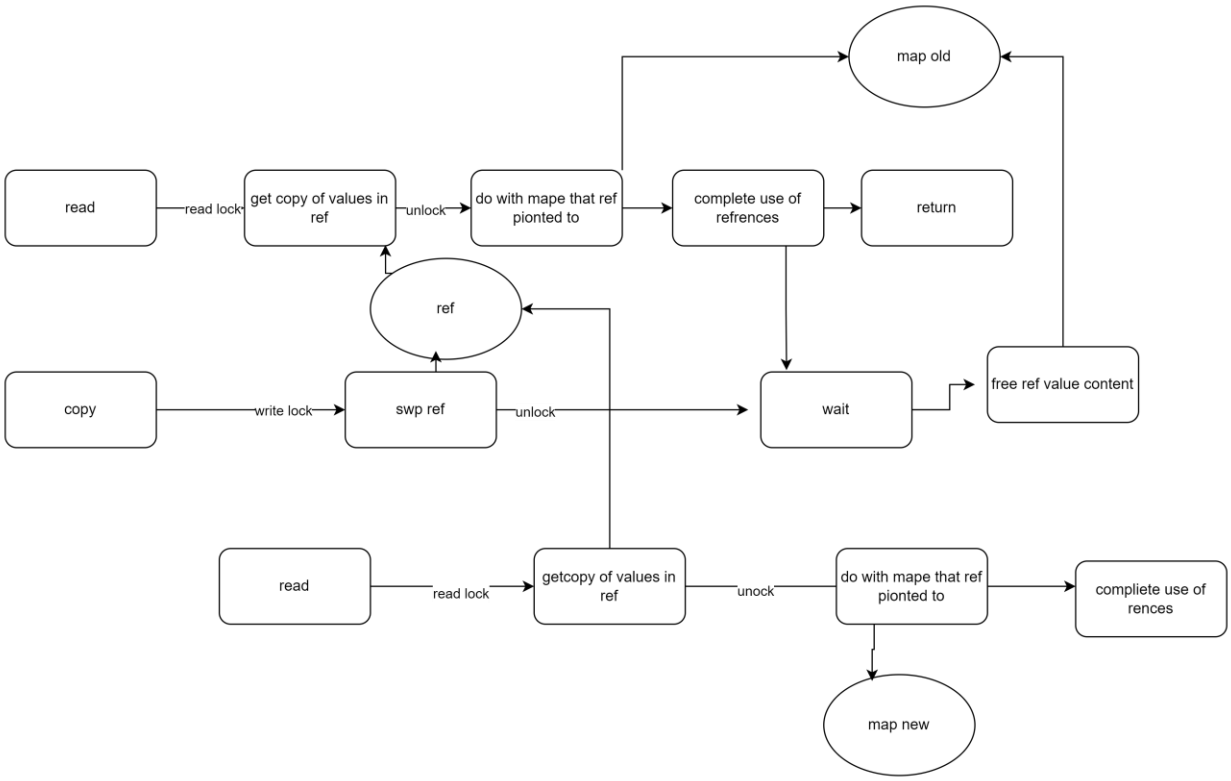
This is a description of the more complicated lock and unlock mechanics for my concurrent hash map.

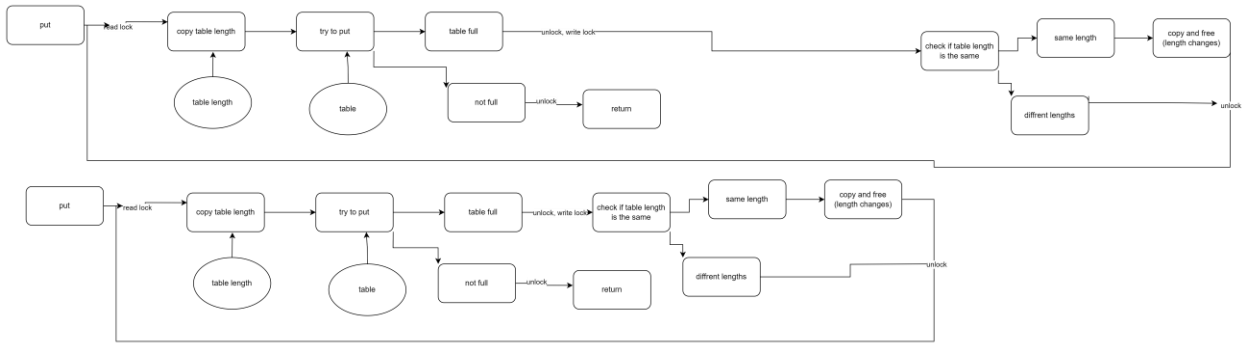
The fist is the interaction between reads and resizing of the HashMap the goal is to minimize down time for reading so reads are only locked out for a quick swape of the pointer to the new table. Read locks are only there for the coping of the data therefore to free from memory the old table we need to be freed to do new writes. and resizing does not need to be written for reads in fact and resizing will happen concurrently.

The freeing function uses a spine lock this was done becase read functions tend to be fast so I don't think its worth sleeping the thread or using the regular cost of locks and the system calls.




The next is the concurrent put calls while concurrent puts are fine while the table is mid resizing all other puts are lock out and becase resizing happened with in the put function there will be no put operation while resizing. This is becase if you have a concurrent put at resizing time it can easily be lost. This is also assumed that if a resizing is need then there no space for additional operations.  Thuse  when putting it will go into a read lock but during

resizing it will switch that read lock to a write lock to prevent any other puts from adding or attempting to do a resize of its own. The first thing a put will do after it acquires the write lock it will then check if the table size if it is different then the old size it means anther put got to the point of resizing and has don't the resize there for there no need for this put to do the resize. This put will then release the lock and try to do the put operation again with the new size set. If the size of the table is the same after aquiline the write lock this means that it need to be resized thus it will do the copy operation and then release the lock.




The last lock is the main lock is the global lock. This is just to make sure that when you are destroying a table there no active operations. Thuse all oration will acquire the read lock then it will check is the table is null if it is it will resiles the table and immediately release the read lock. A destroy call will first acquire the write lock if table is null it will release the lock and return. If it is not null it will go about destroying the table. Then it will set the table as null then relies the lock and return.